

triloop & picoacq

Analysis Code Reference

Pipeline math, module-by-module walkthrough, and per-program description

May 23, 2026

Version 0.1.0

Contents

1	Introduction	1
2	Pipeline overview	1
3	Acquisition: <code>picoacq</code>	2
3.1	<code>picoacq.cli</code>	2
3.2	<code>picoacq.ps5444a</code>	2
3.3	<code>picoacq.simulator</code>	3
3.4	<code>picoacq.recorder</code>	3
3.5	<code>picoacq.alias</code>	3
4	The capture file format	3
5	Geometry and direction transforms	4
5.1	<code>triloop.geometry</code>	4
6	Band extraction and analysis	5
6.1	<code>triloop.extract</code>	5
6.2	<code>triloop.bands</code>	5
6.3	<code>triloop.analyze</code>	5
6.4	<code>triloop.multiband</code>	6
7	Direction finding	6
7.1	<code>triloop.direction</code>	6
7.2	Why find the null instead of the maximum?	6
7.3	<code>triloop.beamform</code>	7
7.4	<code>triloop.stokes</code>	7
8	Reporting and visualization	7
8.1	<code>triloop.view</code>	7
8.2	<code>triloop.browse</code>	7

9	Module roll call	8
9.1	picoacq	8
9.2	triloop	9
10	Tracing a capture through the code	9
11	Testing and verification	10

1 Introduction

This document is a developer’s reference for the analysis code in `triloop` and the acquisition code in `picoacq`. It complements the *triloop User Manual* (which describes how to *use* the tools) and the *picoacq User Manual* (which describes the hardware and the capture file format). Here we focus on *what each Python module does*, in what order they are called by the public entry points, and the mathematics each step implements.

Audience. Reader is comfortable with Python, complex baseband signal processing, and standard linear-algebra notation.

Conventions used in this document. Bold lower-case (**b**) denotes a vector; bold upper-case (**N**) denotes a matrix; $\hat{\mathbf{k}}$ is the unit propagation vector *toward the source*; angles are in degrees in code, radians in equations.

2 Pipeline overview

A capture flows through the toolchain in four stages:

1. **Acquisition** (`picoacq`). Hardware capture orchestrated by `picoacq.recorder.capture()`, which calls the PicoSDK wrapper `picoacq.ps5444a` or, in `--simulate` mode, `picoacq.simulator`. Output: a `triloop`-format HDF5 file with float32 samples in volts plus a JSON `capture_settings` dict carrying the chosen sample rate, voltage ranges, auto-range probe values, and (new in v0.1) the list of expected RF bands `rf_bands_hz`.
2. **Band extraction** (`triloop.bands`, `triloop.extract`). Read the HDF5 file with `triloop.io.hdf5.read_capture`, identify the baseband alias position of each RF band via `picoacq.alias`; mix-and-LPF each channel down to a complex baseband centred on that band; decimate to a workable IF rate.
3. **Polarimetry and direction finding** (`triloop.analyze`, `triloop.geometry`, `triloop.stokes`, `triloop.direction`, `triloop.beamform`). Recover the lab-frame **B**-vector from the three loops via the inverse of the geometry matrix **N**, project onto the plane perpendicular to $\hat{\mathbf{k}}$, compute Stokes parameters and intensity, and (optionally) refine $\hat{\mathbf{k}}$ with a null-eigenvector / null-sweep search.
4. **Reporting** (`triloop.view`, `triloop.multiband`, `triloop.browse`). Static QC plots, multi-band comparison figures, and an interactive single-file HTML browser for visual inspection.

The CLI tools (`triloop view`, `triloop analyze-multi`, `triloop browse`, `triloop analyze`, `triloop locate`, `triloop batch`, `triloop summary`, `triloop simulate`, `triloop notebook`) wrap these stages in different combinations.

3 Acquisition: **picoacq**

3.1 **picoacq.cli**

Click-based command-line interface with two subcommands: **capture** (one-shot) and **monitor** (continuous). Each exposes the same configuration knobs as the underlying recorder; the defaults reflect the multi-band undersampling workflow:

- `--rate 15625000` (Hz)
- `--duration 8.0` (s)
- `--channels A,B,C,D`
- `--range 2.0` (V; only used when `--no-auto-range`)
- `--auto-range/--no-auto-range` (default on)
- `--headroom 3.0`
- `--probe 0.1` (auto-range probe duration, s)
- `--rf-bands "2.5e6,5e6,10e6,15e6,20e6"`

3.2 **picoacq.ps5444a**

A thin ctypes wrapper around the PicoSDK 5000a driver. Five functions:

- `open_5444(n_channels=4)` – context manager that opens the device, automatically picking 16-bit resolution for ≤ 2 channels and 14-bit for 3–4 channels (5444D constraint).
- `configure_channel(ps, handle, ch, range_volts, coupling)` – enables a channel and sets its $\pm$ range.
- `get_timebase(ps, handle, target_rate)` – searches the Pico timebase index space for the closest valid sample rate at or below the requested target. Skips early indices that are invalid in 4-channel mode.
- `acquire_block(ps, handle, channels, n, tb, ranges)` – block-mode acquisition with explicit ctypes-wrapped arguments to avoid signature mismatches between picosdk versions.
- `_ensure_pico_dyld_path()` – macOS-only shim that injects the framework library path into DYLD_LIBRARY_PATH before importing picosdk.

3.3 **picoacq.simulator**

The no-hardware fallback. Generates a 3-loop synthetic dataset with configurable arrival direction, polarization, additive Gaussian noise, and Faraday rotation rate. Used by `--simulate` mode and as the silent fallback when the SDK or hardware is unavailable.

3.4 **picoacq.recorder**

The orchestrator. Top-level function: `capture(output_file, duration_s, sample_rate, channels, ranges_volts, coupling, loops_config, simulate, sim_kwargs, auto_range_enabled, headroom, probe_duration_s, rf_bands, verbose)`. Internally:

1. `_check_buffer()` validates that $\text{duration} \times \text{rate}$ samples fit in the onboard memory (128 MS/ch in 4-channel mode); raises `ValueError` with a clear maximum if not.
2. If auto-range is on (and `ranges_volts` not supplied): open the scope at the maximum ± 20 V on every channel, run a brief probe capture (`auto_range()`), measure peak/RMS in volts, and call `_pick_range()` to choose the smallest available Pico range whose full scale $\geq K \cdot \text{peak}$ (default $K = 3$).
3. Run the science capture via `_try_real_capture()`; on any SDK/hardware failure, fall back transparently to `simulate_capture()`.
4. Convert int16 ADC codes $\rightarrow$ float32 volts; write to HDF5 via `triloop.io_hdf5.write_capture()` with the chosen ranges, the auto-range probe results, and `rf_bands_hz` stored in `capture_settings`.

3.5 picoacq.alias

Pure-Python helper that maps RF frequencies to baseband alias positions for a given sample rate. An RF tone at f_{rf} folds to baseband as

$$f_{\text{bb}} = |f_{\text{rf}} - n f_s|, \quad n = \text{round}(f_{\text{rf}}/f_s), \quad (1)$$

with Nyquist zone $z = \lfloor 2f_{\text{rf}}/f_s \rfloor + 1$. Even zones produce a frequency-inverted spectrum. Three exports:

- Alias dataclass with fields (`rf_hz`, `baseband_hz`, `nyquist_zone`, `inverted`).
- `alias_of(rf_hz, sample_rate_hz)` – single mapping.
- `rf_for_baseband(f_bb, rf_bands, fs, tol)` – inverse lookup (given a detected baseband peak, return which RF band it most likely came from).

This module is the single source of truth for alias bookkeeping; `triloop.bands` and `triloop.view/browse` all import from it.

4 The capture file format

Both packages read and write the same HDF5 v0.1 layout:

```
/
HDF5 root, @format_version = "0.1"
/metadata
  group, attributes:
    @sample_rate      float64, Hz
    @duration_s       float64
    @start_time_utc   ISO-8601
    @scope_model      string
    @scope_serial     string
    @capture_settings JSON-encoded dict
    @loops_config     JSON-encoded dict
/channels
  /A  /B  /C  /D      float32 (or int16) datasets, length N
  each has:
    @gain_v_per_count float64
    @offset_v         float64
/time
optional float64 array, length N
```

`capture_settings` fields recorded by `picoacq.recorder.capture()`:

- `requested_duration_s`, `requested_sample_rate`
- `channels`, `ranges_volts`, `coupling`
- `used_simulator`, `auto_range_enabled`, `headroom`
- `auto_range_probe` (`peaks_volts`, `rms_volts`, `headroom`, `probe_duration_s`) – only when probe ran
- `picoscope_ranges_volts` – the actual chosen ranges
- `rf_bands_hz` – list of expected RF frequencies in Hz
- `truth` – only present in simulator mode

`triloop.io_hdf5` provides `read_capture()` and `write_capture()`; both are short and tested, and any third party can interoperate by writing the same layout.

5 Geometry and direction transforms

5.1 `triloop.geometry`

Operates on the loop-array geometry config dict (`loops_config`). Five public functions:

- `az_el_to_khat(az_deg, el_deg)` returns $\hat{\mathbf{k}} = (\cos \theta \sin \varphi, \cos \theta \cos \varphi, \sin \theta)^\top$ in the local ENU frame.
- `build_N_matrix(loops_config)` returns the 3×3 matrix $\mathbf{N}$ whose i -th row is the i -th loop's normal $\hat{\mathbf{n}}_i$ (in ENU, scaled by per-loop gain). This is the matrix that maps the lab-frame $\mathbf{B}$ to the loop-projected real signal $\mathbf{b} = \mathbf{N}\mathbf{B}$.
- `perp_projector(khat)` returns $\mathbf{P}_\perp = \mathbf{I} - \hat{\mathbf{k}}\hat{\mathbf{k}}^\top$, the projector onto the plane perpendicular to $\hat{\mathbf{k}}$.
- `perp_orthonormal_basis(khat)` returns two unit vectors $(\hat{\mathbf{p}}, \hat{\mathbf{q}})$ that span that plane, with $\hat{\mathbf{p}}$ chosen along the local horizontal whenever possible, $\hat{\mathbf{q}} = \hat{\mathbf{k}} \times \hat{\mathbf{p}}$.
- `LOOP_NORMALS_DEFAULT` and `_normal_from_az_el` support the default cube-vertex layout.

The default geometry is three loops on three faces of a cube whose body diagonal is vertical, giving normal-tilts of $\arctan(1/\sqrt{2}) \approx 35.26^\circ$ above horizontal at azimuths $\{0, +120, -120\}^\circ$.

6 Band extraction and analysis

6.1 `triloop.extract`

Two functions covering the single-band path:

- `extract_complex_baseband(t, b, f0, BW)` performs an FFT-based peak search around f_0 over a $\pm \text{search_BW}$ window, mixes the strongest peak to DC, brick-wall low-passes to $\pm \text{BW}/2$, and returns the complex baseband at the original sample rate, the detected peak frequency, and an FFT-domain SNR estimate.
- `extract_three_loops(...)` calls the above on each loop, applies per-loop `phase_offset_deg` calibration, and returns a stacked $(3, N)$ complex array.

6.2 triloop.bands

Multi-band counterpart, used when the capture's `rf_bands_hz` populates more than one entry. Public function: `extract_bands(cap, rf_bands, bw_hz, decim_rate_hz, channels)`. For every band:

1. `picoacq.alias.alias_of()` computes f_{bb} , the Nyquist zone, and `inverted`.
2. Validate that the requested `bw_hz` doesn't reach into the adjacent Nyquist edge or another band's alias.
3. Mix every selected channel down to f_{bb} via FFT (`_extract_one_channel`), brick-wall filter to $\pm bw_hz/2$, decimate to the smallest integer factor that yields $\geq decim_rate_hz$, and conjugate the result if the band lives in an even (inverted) Nyquist zone.

Returns a dict mapping `rf_hz` $\rightarrow$ `BandExtraction` dataclass with fields `rf_hz`, `baseband_hz`, `nyquist_zone`, `inverted`, `bw_hz`, `decim_rate_hz`, `t`, `z`, where `z` is a per-channel dict of `complex64` arrays.

6.3 triloop.analyze

The polarimetry pipeline. Two public entry points:

- `analyze(t, B1, B2, B3, f0, BW, az_deg, el_deg, loops_config)` – starts from real loop signals, calls `extract_three_loops()`, then forwards to:
- `analyze_z_loops(t, Z_loops, f_peak, snrs_db, az_deg, el_deg, loops_config)` – the inner pipeline, also called directly by the multi-band path.

The inner pipeline computes:

$$\mathbf{z}_{lab}(t) = \mathbf{N}^{-1} \mathbf{z}_{loops}(t), \quad (2)$$

$$\mathbf{z}_{\perp}(t) = \mathbf{P}_{\perp} \mathbf{z}_{lab}(t), \quad (3)$$

$$I(t) = |\mathbf{z}_{\perp}|^2, \quad (4)$$

$$A_p(t) = \hat{\mathbf{p}} \cdot \mathbf{z}_{\perp}, \quad A_q(t) = \hat{\mathbf{q}} \cdot \mathbf{z}_{\perp}, \quad (5)$$

then forwards (A_p, A_q) to `stokes.compute_stokes()` which returns the four Stokes parameters, the polarization fraction $\sqrt{Q^2 + U^2 + V^2}/I$, the ellipticity $\frac{1}{2} \arcsin(V/I_p)$ in degrees ($\pm 45^\circ$ at full circular polarization), and the position angle $\frac{1}{2} \arctan 2(U, Q)$.

A 2×2 coherency on (A_p, A_q) identifies the dominant-pol eigenvector u_{dom} , and $z_{dom}(t) = u_{dom}^\top(A_p, A_q)$ provides an instantaneous phase signal whose unwrapped, linearly-detrended residual gives both the residual carrier frequency and a time-resolved frequency series via `numpy.gradient`. Everything is bundled into an `AnalysisResult` dataclass.

6.4 triloop.multiband

Wraps band extraction and per-band analysis. Public functions:

- `analyze_all_bands(cap, az_deg, el_deg, loops_channels, bw_hz, decim_rate_hz, rf_bands, loops_config)` – calls `extract_bands` once and runs `analyze_z_loops` on each band. Returns a dict mapping `rf_hz` $\rightarrow$ `MultiBandResult` with fields `(rf_hz, extraction, analysis, snrs_db)`.

- `make_multiband_figure(results, out_path)` – plot a five-row, four-column matplotlib figure (one row per band) of intensity, instantaneous frequency, ellipticity, and polarization fraction.

7 Direction finding

7.1 `triloop.direction`

Public functions, all operating on a stacked lab-frame complex array $Z_{\text{lab}} \in \mathbb{C}^{3 \times N}$:

- `coherency_matrix(Z_lab)` returns $C = \frac{1}{N} Z_{\text{lab}} Z_{\text{lab}}^H$, the 3×3 Hermitian coherency.
- `estimate_direction_from_z_lab(Z_lab)` returns the smallest-eigenvalue eigenvector of C as $\hat{\mathbf{k}}_{\text{est}}$, plus the polarization basis ($\hat{\mathbf{p}}$, $\hat{\mathbf{q}}$ from the two largest eigenvectors), the eigenvalues (descending), and a scalar `null_strength` λ_3/λ_1 . This is the *coarse* direction estimate; small `null_strength` means the rank-1 trap (§ 7.2) is broken and the estimate is trustworthy.
- `perp_residual_energy(Z_lab, khat)` computes $\langle |\hat{\mathbf{k}} \cdot \mathbf{z}_{\text{lab}}|^2 \rangle$, the quantity minimised by direction finding.
- `null_search(Z_lab, az0, el0, half_width_deg, n_points)` performs a 2-D grid sweep around (az_0, el_0) and returns the minimiser plus the full residual map.
- `parabolic_refine(R, az_grid, el_grid)` fits a parabola to the 3×3 neighbourhood of the grid minimum to give a sub-grid direction.
- `lock_and_analyze(...)` – end-to-end entry point used by the `locate` CLI.

7.2 Why find the null instead of the maximum?

The direction-formed power $P(\hat{\mathbf{k}}) = \langle |\mathbf{B}_{\perp}(\hat{\mathbf{k}})|^2 \rangle$ varies as $\frac{1}{2} + \frac{1}{2} \cos^2 \beta$ near the true direction $\hat{\mathbf{k}}_{\text{true}}$ (a shallow plateau). The *perpendicular* residual $Q(\hat{\mathbf{k}}) = \langle |\hat{\mathbf{k}} \cdot \mathbf{B}|^2 \rangle$ varies as $\sin^2 \beta \sim \beta^2$ near the same direction (a sharp parabolic minimum). Localising a parabolic minimum is far more precise than a cosine-squared maximum, so we always lock on the null rather than the peak.

For purely linearly polarized signals from a single direction, C is rank 1 and the null is a 2-D plane rather than a point. In that regime `estimate_direction_from_z_lab` returns a direction (the smallest-eigenvalue one within floating-point noise), but it is not physically meaningful; trustworthy localisation requires Faraday rotation, a circularly polarized source, or triangulation.

7.3 `triloop.beamform`

Older, simpler beamforming sweep used by some analysis notebooks. Two functions:

- `beamform_grid(z_loops, loops_config, az_grid_deg, el_grid_deg)` – compute time-averaged $P(\varphi, \theta) = \langle |\mathbf{B}_{\perp}(\hat{\mathbf{k}}_{\varphi, \theta})|^2 \rangle$ over a 2-D grid.
- `best_direction(P, az_grid, el_grid)` – return the arg max as (φ, θ) .

7.4 `triloop.stokes`

A single function: `compute_stokes(A_p, A_q)` returning a dict with `I`, `Q`, `U`, `V`, `pol_fraction`, `ellipticity_deg`, `position_angle_deg`. Definitions: $I = |A_p|^2 + |A_q|^2$, $Q = |A_p|^2 - |A_q|^2$, $U = 2\Re\{A_p^* A_q\}$, $V = 2\Im\{A_p^* A_q\}$.

8 Reporting and visualization

8.1 `triloop.view`

Static QC PNG generator used by `triloop.view`. `make_view_figure(cap, rf_bands, bw_hz, out_path)` produces a single matplotlib figure with three rows: a full-Nyquist PSD with each RF band annotated at its alias position, per-band PSD zooms (with the displayed f-axis flipped for inverted bands so the operator reads frequencies the right way), a decimated time-domain trace of every channel, and the auto-range probe table.

8.2 `triloop.browse`

Interactive single-file HTML browser. Top-level entry points:

- `build_browser_html(cap, az_deg, el_deg, loops_channels, bw_hz, decim_rate_hz, n_anim_frames, title)` – returns a self-contained HTML string.
- `write_browser(cap, out_path, **kw)` – save it to disk.

The page uses Plotly (loaded once via CDN; if you need a fully offline build, change the `include_plotlyjs="cdn"` argument to `True` and the page will inline the entire library at the cost of ~ 3 MB extra). Layout: tab bar at the top with *Overview*, *Time series*, and one tab per RF band. The per-band tab has four panels: PSD zoom, intensity time history (linear + togglable dB), intensity CDF with P10/P1 fade-depth markers, and an animated polarization ellipse with Play/Pause and a frame slider.

The animation is implemented as Plotly frames; each frame draws a sliding window of $(\Re A_p, \Re A_q)$ samples centred on a different slow-time epoch. The window length is chosen automatically to be a few cycles of the residual carrier-phase variation at the decimated rate.

9 Module roll call

This section lists every Python file in both packages and what each one provides. Treat it as a table of contents for the source.

9.1 picoacq

Module	Purpose
<code>__init__.py</code>	package marker; no public API of its own.
<code>cli.py</code>	Click-based CLI; subcommands <code>capture</code> and <code>monitor</code> . Reads <code>--rate</code> / <code>--duration</code> / <code>--channels</code> / <code>--range</code> / <code>--auto-range</code> / <code>--headroom</code> / <code>--probe</code> / <code>--rf-bands</code> / <code>--simulate</code> / <code>--sim-*</code> options and forwards them to <code>recorder.capture()</code> .
<code>recorder.py</code>	Top-level <code>capture()</code> orchestrator with <code>_check_buffer()</code> guard, <code>auto_range()</code> probe, range chooser <code>_pick_range()</code> , and the <code>_try_real_capture()</code> hardware path (with simulator fallback).
<code>ps5444a.py</code>	PicoSDK 5000a wrapper: <code>open_5444()</code> , <code>configure_channel()</code> , <code>get_timebase()</code> , <code>acquire_block()</code> . Plus <code>_ensure_pico_dyld_path()</code> for macOS framework search.
<code>simulator.py</code>	<code>simulate_capture()</code> – 3-loop no-hardware fallback compatible with the <code>recorder</code> return shape.
<code>alias.py</code>	RF $\leftrightarrow$ baseband mapping under bandpass undersampling. Public: <code>Alias</code> dataclass, <code>alias_of()</code> , <code>aliases_for_bands()</code> , <code>rf_for_baseband()</code> . See Section 3.5.

9.2 triloop

Module	Purpose
<code>__init__.py</code>	re-exports the public API (see <i>User Manual</i> for the full list).
<code>cli.py</code>	Click-based CLI; subcommands <code>view</code> , <code>analyze-multi</code> , <code>browse</code> , <code>analyze</code> , <code>locate</code> , <code>batch</code> , <code>summary</code> , <code>simulate</code> , <code>notebook</code> .
<code>io_hdf5.py</code>	<code>read_capture()</code> and <code>write_capture()</code> for the HDF5 v0.1 layout described in §4.
<code>config.py</code>	<code>default_loops_config()</code> (cube-vertex geometry) and <code>validate_loops_config()</code> schema check.
<code>geometry.py</code>	<code>az_el_to_khat()</code> , <code>build_N_matrix()</code> , <code>perp_projector()</code> , <code>perp_orthonormal_basis()</code> , <code>LOOP_NORMALS_DEFAULT</code> .
<code>extract.py</code>	Single-band complex-baseband extractor: <code>extract_complex_baseband()</code> , <code>extract_three_loops()</code> .
<code>bands.py</code>	Multi-band extractor: <code>extract_bands()</code> , <code>BandExtraction</code> dataclass.
<code>stokes.py</code>	<code>compute_stokes(A_p, A_q)</code> -> Stokes parameters and derived quantities.
<code>analyze.py</code>	Top-level <code>analyze()</code> and inner <code>analyze_z_loops()</code> ; <code>AnalysisResult</code> dataclass.
<code>multiband.py</code>	<code>analyze_all_bands()</code> (extract + analyze for every band) and <code>make_multiband_figure()</code> .
<code>direction.py</code>	Coherency-based direction finding: <code>coherency_matrix()</code> , <code>estimate_direction_from_z_lab()</code> , <code>perp_residual_energy()</code> , <code>null_search()</code> , <code>parabolic_refine()</code> , <code>lock_and_analyze()</code> .
<code>beamform.py</code>	Older/simpler grid-sweep: <code>beamform_grid()</code> , <code>best_direction()</code> .
<code>view.py</code>	Static QC PNG: <code>make_view_figure()</code> .
<code>browse.py</code>	Interactive Plotly HTML browser: <code>build_browser_html()</code> , <code>write_browser()</code> , plus the helpers <code>_channel_psd()</code> , <code>_band_zoom_spectrum()</code> , <code>_smooth_decimate()</code> , <code>_cdf()</code> , <code>_build_pol_animation()</code> .

10 Tracing a capture through the code

To consolidate the picture above, here are the call paths for the three new commands:

`triloop view cap.h5`.

1. `cli.view_cmd` parses options.
2. `io_hdf5.read_capture(cap.h5)` -> dict.
3. `view.make_view_figure(cap, rf_bands, bw_hz, out_path)`.
4. Inside `make_view_figure`: per-channel PSD via `_channel_psd`; per-band zoom via `_band_zoom_spectrum` (calls `picoacq.alias.alias_of` for each band centre); time-domain decimated; probe table from `capture_settings.auto_range_probe`.

`triloop analyze-multi cap.h5 --az 9 --el 35`.

1. `cli.analyze_multi_cmd` parses options.

2. `io_hdf5.read_capture(cap.h5)` -> dict.
3. `multiband.analyze_all_bands(cap, az, el, ...)`:
 - (a) `bands.extract_bands(cap, ...)` which for each band uses `picoacq.alias.alias_of` to find f_{bb} , then mixes and decimates each channel via `_extract_one_channel`.
 - (b) For each band: `_per_loop_snr_from_baseband`, then `analyze.analyze_z_loops(t, Z_loops, f_rf, ...)`.
4. `multiband.make_multiband_figure(results, out_png)`.
5. Per-band JSON summary written to `out_json`.

`triloop` browse `cap.h5`.

1. `cli.browse_cmd` parses options.
2. `io_hdf5.read_capture(cap.h5)` -> dict.
3. `browse.write_browser(cap, out_html, ...)` which calls `build_browser_html(cap, ...)`:
 - (a) Calls `multiband.analyze_all_bands` (same path as above) to populate per-band intensity, ellipticity, and A_p/A_q time series.
 - (b) Builds per-tab Plotly figures: overview PSD, time series, and one 4-panel tab per band.
 - (c) For each band assembles a polarization-ellipse animation via `_build_pol_animation()` (Plotly frames + slider).
 - (d) Renders the page using `plotly.io.to_html()` for each figure into a tabbed HTML scaffold.

11 Testing and verification

The test suite under `triloop/tests/` exercises the analysis pipeline against the simulator at known polarizations and directions. Useful commands during development:

```
# Create a synthetic 4-channel capture with all 5 WWV bands aliased
PYTHONPATH=triloop:picoacq python3 -m picoacq.cli capture \
    --simulate --duration 0.5 -o sim.h5

# Run the multi-band analysis and confirm pol_frac and ellipticity
PYTHONPATH=triloop:picoacq python3 -m triloop.cli analyze-multi sim.h5 \
    --az 9 --el 35

# Render the interactive browser
PYTHONPATH=triloop:picoacq python3 -m triloop.cli browse sim.h5
```

For the end-to-end “does the alias arithmetic come out right” check on a synthetic 4-channel 15.625 MS/s capture, see `triloop/triloop/bands.py` (specifically `extract_bands` will raise `ValueError` if the requested per-band bandwidth overlaps the Nyquist edge or another band’s alias, which is the nearest thing to an automatic correctness check).